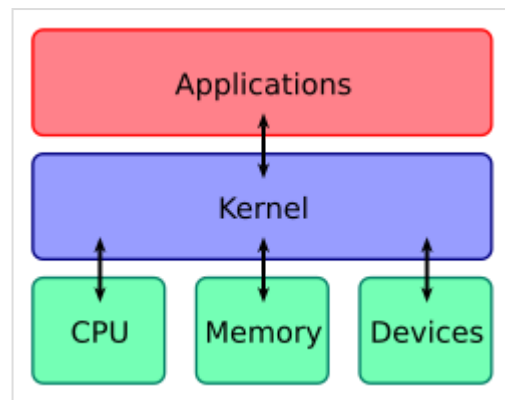


Kernel (operating system)

A **kernel** is a computer program at the core of a computer's operating system that always has complete control over everything in the system. The kernel is also responsible for preventing and mitigating conflicts between different processes.^[1] It is the portion of the operating system that is always resident in memory^[2] and facilitates interactions between hardware and software components. A full kernel controls all hardware resources (e.g. I/O, memory, cryptography) via device drivers, arbitrates conflicts between processes concerning such resources, and optimizes the use of common resources, such as CPU, cache, file systems, and network sockets. On most systems, the kernel is one of the first programs loaded on startup (after the bootloader). It handles the rest of startup as well as memory, peripherals, and input/output (I/O) requests from software, translating them into data-processing instructions for the central processing unit.



A simplification of how a kernel connects application software to the hardware of a computer

The critical code of the kernel is usually loaded into a separate area of memory, which is protected from access by application software or other less critical parts of the operating system. The kernel performs its tasks, such as scheduling processes, managing hardware devices such as the hard disk, and handling interrupts, in this protected kernel space. In contrast, application programs such as browsers, word processors, or audio or video players use a separate area of memory, user space. This prevents user data and kernel data from interfering with each other and causing instability and slowness,^[1] as well as preventing malfunctioning applications from affecting other applications or crashing the entire operating system. Even in systems where the kernel is included in application address spaces, memory protection is used to prevent unauthorized applications from modifying the kernel.

The kernel's interface is a low-level abstraction layer. When a process requests a service from the kernel, it must invoke a system call, usually through a wrapper function.

There are different kernel architecture designs. Monolithic kernels run entirely in a single address space with the CPU executing in supervisor mode, mainly for speed. Microkernels run most but not all of their services in user space,^[3] like user processes do, mainly for resilience and modularity.^[4] MINIX 3 is a notable example of microkernel design. Some kernels, such as the Linux kernel, are both monolithic and modular, since they can insert and remove loadable kernel modules at runtime.

This central component of a computer system is responsible for executing programs. The kernel takes responsibility for deciding at any time which of the many running programs should be allocated to the processor or processors.

Random-access memory

Random-access memory (RAM) is used to store both program instructions and data.^[a] Typically, both need to be present in memory for a program to execute. Often, multiple programs will want memory access, frequently demanding more memory than the computer has available. The kernel is responsible for deciding which memory each process can use, and determining what to do when insufficient memory is available.

Input/output devices

I/O devices include, but are not limited to, peripherals such as keyboards, mice, disk drives, printers, USB devices, network adapters, and display devices. The kernel provides convenient methods for applications to use these devices, which are typically abstracted by the kernel so that applications do not need to know their implementation details.

Resource management

Key aspects necessary in resource management are defining the execution domain (address space) and the protection mechanism used to mediate access to the resources within a domain.^[5] Kernels also provide methods for synchronization and inter-process communication (IPC). These implementations may be located within the kernel itself or the kernel can also rely on other processes it is running. Although the kernel must provide IPC in order to provide access to the facilities provided by each other, kernels must also provide running programs with a method to make requests to access these facilities. The kernel is also responsible for context switching between processes or threads.

Memory management

The kernel has full access to the system's memory and must allow processes to safely access this memory as they require it. Often the first step in doing this is virtual addressing, usually achieved by paging and/or segmentation. Virtual addressing allows the kernel to make a given physical address appear to be another address, the virtual address. Virtual address spaces may be different for different processes; the memory that one process accesses at a particular (virtual) address may be different memory from what another process accesses at the same address. This allows every program to behave as if it is the only one (apart from the kernel) running and thus prevents applications from crashing each other.^[6]

On many systems, a program's virtual address may refer to data that is not currently in memory. The layer of indirection provided by virtual addressing allows the operating system to use other data stores, like a hard drive, to store what would otherwise have to remain in main memory (RAM). As a result, operating systems can allow programs to use more memory than the system has physically available. When a program needs data that is not currently in RAM, the CPU signals to the kernel that this has happened, and the kernel responds by writing the contents of an inactive memory block to disk (if necessary) and replacing it with the data requested by the program. The program can then be resumed from the point where it was stopped. This scheme is generally known as demand paging.

Virtual addressing also allows creation of virtual partitions of memory in two disjoint areas, one being reserved for the kernel (kernel space) and the other for the applications (user space). The applications are not permitted by the processor to address kernel memory, thus preventing an application from damaging the running kernel. This fundamental partition of memory space has contributed much to the current designs of actual general-purpose kernels and is almost universal in such systems, although some research kernels (e.g., Singularity) take other approaches.

Device management

To perform useful functions, processes need access to the peripherals connected to the computer, which are controlled by the kernel through device drivers. A device driver is a computer program encapsulating, monitoring and controlling a hardware device (via its hardware/software interface (HSI) (<https://courses.cs.washington.edu/courses/cse351/11wi/lectures/>)) on behalf of the OS. It provides the operating system with an API, procedures and information about how to control and communicate with a certain piece of hardware. Device drivers are an important and vital dependency for all OS and their applications. The design goal of a driver is abstraction; the function of the driver is to translate the OS-mandated abstract function calls (programming calls) into device-specific calls. In theory, a device should work correctly with a suitable driver. Device drivers are used for e.g. host adapters, video cards, sound cards, printers, scanners, modems, and network interface controllers.

At the hardware level, common abstractions of device drivers include:

- Interfacing directly
- Using a high-level interface, such as a Video BIOS
- Using a lower-level device driver (file system drivers using disk drivers)
- Simulating work with hardware, while doing something entirely different

And at the software level, device driver abstractions include:

- Allowing the operating system direct access to hardware resources
- Only implementing primitives
- Implementing an interface for non-driver software such as TWAIN
- Implementing a language (often a high-level language such as PostScript)

For example, to show the user something on the screen, an application would make a request to the kernel, which would forward the request to its display driver, which is then responsible for actually plotting the character/pixel.^[6]

A kernel must maintain a list of available devices. This list may be known in advance (e.g., on an embedded system where the kernel will be rewritten if the available hardware changes), configured by the user (typical on older PCs and on systems that are not designed for personal use) or detected by the operating system at run time (normally called plug and play). In plug-and-play systems, a device manager first performs a scan on different peripheral buses, such as Peripheral Component Interconnect (PCI) or Universal Serial Bus (USB), to detect installed devices, then searches for the appropriate drivers.

As device management is a very OS-specific topic, these drivers are handled differently by each kind of kernel design, but in every case, the kernel has to provide the I/O to allow drivers to physically access their devices through some port or memory location. Important decisions have to be made when

designing the device management system, as in some designs accesses may involve context switches, making the operation very CPU-intensive and easily causing a significant performance overhead.

System calls

In computing, a system call is how a process requests a service from an operating system's kernel that it does not normally have permission to run. System calls provide the interface between a process and the operating system. Most operations interacting with the system require permissions not available to a user-level process, e.g., I/O performed with a device present on the system, or any form of communication with other processes requires the use of system calls.

A system call is a mechanism that is used by the application program to request a service from the operating system. They use a machine-code instruction that causes the processor to change mode. An example would be from supervisor mode to protected mode. This is where the operating system performs actions like accessing hardware devices or the memory management unit. Generally the operating system provides a library that sits between the operating system and normal user programs. Usually it is a C library such as Glibc or Windows API. The library handles the low-level details of passing information to the kernel and switching to supervisor mode. System calls include close, open, read, wait and write.

To actually perform useful work, a process must be able to access the services provided by the kernel. This is implemented differently by each kernel, but most provide a C library or an API, which in turn invokes the related kernel functions.^[7]

The method of invoking the kernel function varies from kernel to kernel. If memory isolation is in use, it is impossible for a user process to call the kernel directly, because that would be a violation of the processor's access control rules. A few possibilities are:

- Using an interrupt.^[e] This method is available on most hardware, and is therefore very common.
- Using a call gate. A call gate is a special address stored by the kernel in a list in kernel memory at a location known to the processor. When the processor detects a call to that address, it instead redirects to the target location without causing an access violation. This requires hardware support, but the hardware for it is quite common.
- Using a special system call instruction. This technique requires special hardware support that common architectures (notably, x86) may lack; for those architectures, interrupts or call gates are used. System call instructions have been added to recent models of x86 processors, however, and some operating systems for PCs make use of them when available.
- Using a memory-based queue. An application that makes large numbers of requests but does not need to wait for the result of each may add details of requests to an area of memory that the kernel periodically scans to find requests.

Kernel design decisions

Protection

An important consideration in the design of a kernel is the support it provides for protection from faults (fault tolerance) and from malicious behaviours (security). These two aspects are usually not clearly distinguished, and the adoption of this distinction in the kernel design leads to the rejection of a hierarchical structure for protection.^[5]

The mechanisms or policies provided by the kernel can be classified according to several criteria, including:

- whether they are static (enforced at compile time) or dynamic (enforced at run time);
- whether they are pre-emptive or post-detection;
- whether they are hardware-supported or language-based;
- whether they are more an open mechanism, or a binding policy;
- according to the protection principles they satisfy (e.g., Denning^{[11][12]});
- and many more.

Support for hierarchical protection domains^[13] is typically implemented using CPU modes.

Many kernels implement "capabilities", i.e., objects provided to user code that allow limited access to an underlying object managed by the kernel. A common example is file handling: a file is a representation of information stored on a permanent storage device. The kernel may be able to perform many different operations, including read, write, delete or execute, but a user-level application may only be permitted to perform some of these operations (e.g., it may only be allowed to read the file). A common implementation of this is for the kernel to provide an object to the application (typically called a "file handle") that the application may then invoke operations on, the validity of which the kernel checks at the time the operation is requested. Such a system may be extended to cover all objects that the kernel manages, and indeed to objects provided by other user applications.

An efficient and simple way to provide hardware support of capabilities is to delegate to the memory management unit (MMU) the responsibility of checking access-rights for every memory access, a mechanism called capability-based addressing.^[14] Most commercial computer architectures lack such MMU support for capabilities; the Capability Hardware Enhanced RISC Instructions (CHERI) project is developing capability-based mechanisms for several architectures.

An alternative approach is to simulate capabilities using commonly supported hierarchical domains. In this approach, each protected object must reside in an address space that the application does not have access to; the kernel also maintains a list of capabilities in such memory. When an application needs to access an object protected by a capability, it performs a system call and the kernel then checks whether the application's capability grants it permission to perform the requested action, and if it is permitted performs the access for it (either directly, or by delegating the request to another user-level process). The performance cost of address space switching limits the practicality of this approach in systems with complex interactions between objects, but it is used in current operating systems for objects that are not accessed frequently or that are not expected to perform quickly.^{[15][16]}

If the firmware does not support protection mechanisms, it is possible to simulate protection at a higher level, for example by simulating capabilities by manipulating page tables, but there are performance implications.^[17] Lack of hardware support may not be an issue, however, for systems that choose to use language-based protection.^[18]

An important kernel design decision is the choice of the abstraction levels where the security mechanisms and policies should be implemented. Kernel security mechanisms play a critical role in supporting security at higher levels.^{[14][19][20][21][22]}

One approach is to use firmware and kernel support for fault tolerance (see above), and build the security policy for malicious behavior on top of that (adding features such as cryptography mechanisms where necessary), delegating some responsibility to the compiler. Approaches that delegate enforcement of security policy to the compiler and/or the application level are often called language-based security.

The lack of many critical security mechanisms in current mainstream operating systems impedes the implementation of adequate security policies at the application abstraction level.^[19] In fact, a common misconception in computer security is that any security policy can be implemented in an application regardless of kernel support.^[19]

According to Mars Research Group developers, a lack of isolation is one of the main factors undermining kernel security.^[23] They propose their driver isolation framework for protection, primarily in the Linux kernel.^{[24][25]}

Hardware- or language-based protection

Typical computer systems today use hardware-enforced rules about what programs are allowed to access what data. The processor monitors the execution and stops a program that violates a rule, such as a user process that tries to write to kernel memory. In systems that lack support for capabilities, processes are isolated from each other by using separate address spaces.^[26] Calls from user processes into the kernel are regulated by requiring them to use one of the above-described system call methods.

An alternative approach is to use language-based protection. In a language-based protection system, the kernel will only allow code to execute that has been produced by a trusted language compiler. The language may then be designed such that it is impossible for the programmer to instruct it to do something that will violate a security requirement.^[18]

Advantages of this approach include:

- No need for separate address spaces. Switching between address spaces is a slow operation that causes a great deal of overhead, and a lot of optimization work is currently performed in order to prevent unnecessary switches in current operating systems. Switching is completely unnecessary in a language-based protection system, as all code can safely operate in the same address space.
- Flexibility. Any protection scheme that can be designed to be expressed via a programming language can be implemented using this method. Changes to the protection scheme (e.g. from a hierarchical system to a capability-based one) do not require new hardware.

Disadvantages include:

- Longer application startup time. Applications must be verified when they are started to ensure they have been compiled by the correct compiler, or may need recompiling either

from source code or from bytecode.

- **Inflexible type systems.** On traditional systems, applications frequently perform operations that are not type safe. Such operations cannot be permitted in a language-based protection system, which means that applications may need to be rewritten and may, in some cases, lose performance.

Examples of systems with language-based protection include JX and Microsoft's Singularity.

Process cooperation

Edsger Dijkstra proved that from a logical point of view, atomic lock and unlock operations operating on binary semaphores are sufficient primitives to express any functionality of process cooperation.^[27] Nonetheless, this approach is generally held to be lacking in terms of safety and efficiency, whereas a message passing approach is more flexible.^[28] A number of other approaches (either lower- or higher-level) are available as well, with many modern kernels providing support for systems such as shared memory and remote procedure calls.

I/O device management

The idea of a kernel where I/O devices are handled uniformly with other processes, as parallel co-operating processes, was first proposed and implemented by Brinch Hansen (although similar ideas were suggested in 1967^{[29][30]}). In Hansen's description of this, the "common" processes are called *internal processes*, while the I/O devices are called *external processes*.^[28]

Similar to physical memory, allowing applications direct access to controller ports and registers can cause the controller to malfunction, or system to crash. With this, depending on the complexity of the device, some devices can get surprisingly complex to program, and use several different controllers. Because of this, providing a more abstract interface to manage the device is important. This interface is normally done by a device driver or hardware abstraction layer. Frequently, applications will require access to these devices. The kernel must maintain the list of these devices by querying the system for them in some way. This can be done through the BIOS, or through one of the various system buses (such as PCI/PCIE, or USB). Using an example of a video driver, when an application requests an operation on a device, such as displaying a character, the kernel needs to send this request to the current active video driver. The video driver, in turn, needs to carry out this request. This is an example of inter-process communication (IPC).

Kernel-wide design approaches

The above listed tasks and features can be provided in many ways that differ from each other in design and implementation.

The principle of *separation of mechanism and policy* is the substantial difference between the two main philosophies of microkernels and monolithic kernels.^{[31][32]} Here a *mechanism* is the support that allows the implementation of many different policies, while a *policy* is a particular "mode of operation". Example:

- **Mechanism:** User login attempts are routed to an authorization server
- **Policy:** Authorization server requires a password that is verified against stored passwords in a database

Because the mechanism and policy are separated, the policy can be easily changed to, e.g., require the use of a security token.

In a minimal microkernel just some very basic policies are included,^[32] and its mechanisms allow what is running on top of the kernel (the remaining part of the operating system and the other applications) to decide which policies to adopt (such as for memory management, high level process scheduling, file system management, etc.).^{[5][28]} A monolithic kernel instead tends to include many policies, therefore restricting the rest of the system to rely on them.

Computer scientist Per Brinch Hansen argued in favour of separation of mechanism and policy.^{[5][28]} The failure to properly fulfill this separation is one of the major causes of the lack of substantial innovation in existing operating systems,^[5] a problem common in computer architecture.^{[33][34][35]} The monolithic design is induced by the "kernel mode"/"user mode" architectural approach to protection (technically called hierarchical protection domains), which is common in conventional commercial systems;^[36] in fact, every module needing protection is therefore preferably included into the kernel.^[36] This link between monolithic design and "privileged mode" can be reconducted to the key issue of mechanism-policy separation;^[5] in fact the "privileged mode" architectural approach melds together the protection mechanism with the security policies, while the major alternative architectural approach, capability-based addressing, clearly distinguishes between the two, leading naturally to a microkernel design.^[5]

While monolithic kernels execute all of their code in the same address space (kernel space), microkernels try to run most of their services in user space, aiming to improve maintainability and modularity of their codebase.^[4] Most kernels do not fit exactly into one of these categories, but are rather found in between these two designs. These are called hybrid kernels. More exotic designs such as nanokernels and exokernels are available, but are seldom used for production systems. The Xen hypervisor, for example, is an exokernel.

Monolithic kernels

In a monolithic kernel, all OS services are part of the kernel and run in kernel mode, thus also residing in the same memory area. This approach provides rich and powerful hardware access. UNIX developer Ken Thompson stated that "it is in [his] opinion easier to implement a monolithic kernel".^[37] The main disadvantages of monolithic kernels are the dependencies between system components – a bug in a device driver might crash the entire system, for example – and the fact that large kernels can become very difficult to maintain. Thompson also stated that "It is also easier for [a monolithic kernel] to turn into a mess in a hurry as it is modified".^[37]

Monolithic kernels, which have traditionally been used by Unix-like operating systems, contain all the operating system core functions and the device drivers. A monolithic kernel is one single program that contains all of the code necessary to perform every kernel-related task.

Every part that is to be accessed by a program that cannot be put in a library is in the kernel space, including drivers, schedulers, memory management, file systems, and network stacks. Many system calls

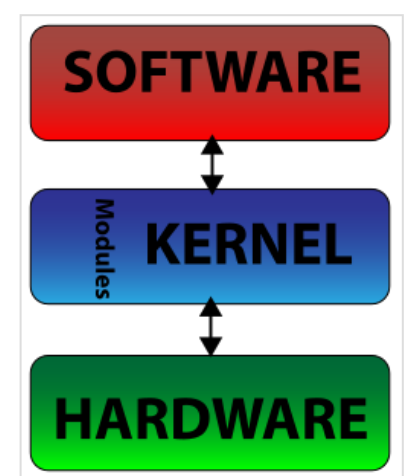


Diagram of a monolithic kernel

are provided to applications, to allow them to access all those services. A monolithic kernel, while initially loaded with subsystems that may not be needed, can be tuned to a point where it is as fast as or faster than one that was specifically designed for the hardware, although more relevant in a general sense.

Modern monolithic kernels, such as the Linux kernel, the FreeBSD kernel, the AIX kernel, the HP-UX kernel, and the Solaris kernel, all of which fall into the category of Unix-like operating systems, support loadable kernel modules, allowing modules to be loaded into the kernel at runtime, permitting easy extension of the kernel's capabilities as required, while helping to minimize the amount of code running in kernel space.

Most work in the monolithic kernel is done via system calls. These are interfaces, usually kept in a tabular structure, that access some subsystem within the kernel, such as disk operations. Essentially, calls are made within programs and a checked copy of the request is passed through the system call. Hence, not far to travel at all. The monolithic Linux kernel can be made extremely small not only because of its ability to dynamically load modules but also because of its ease of customization. In fact, there are some versions that are small enough to fit together with a large number of utilities and other programs on a single floppy disk and still provide a fully functional operating system (one of the most popular of which is muLinux). This ability to miniaturize its kernel has also led to a rapid growth in the use of Linux in embedded systems.

These types of kernels consist of the core functions of the operating system and the device drivers with the ability to load modules at runtime. They provide rich and powerful abstractions of the underlying hardware. They provide a small set of simple hardware abstractions and use applications called servers to provide more functionality. This particular approach defines a high-level virtual interface over the hardware, with a set of system calls to implement operating system services such as process management, concurrency and memory management in several modules that run in supervisor mode. This design has several flaws and limitations:

- Coding in kernel can be challenging, in part because one cannot use common libraries (like a full-featured libc), and because one needs to use a source-level debugger like gdb. Rebooting the computer is often required. This is not just a problem of convenience to the developers. When debugging is harder, and as difficulties become stronger, it becomes more likely that code will be "buggier".
- Bugs in one part of the kernel have strong side effects; since every function in the kernel has all the privileges, a bug in one function can corrupt data structures of another, totally unrelated part of the kernel, or of any running program.
- Kernels often become very large and difficult to maintain.
- Even if the modules servicing these operations are separate from the whole, the code integration is tight and difficult to do correctly.
- Since the modules run in the same address space, a bug can bring down the entire system.

Microkernels

Microkernel (also abbreviated μK or uK) is the term describing an approach to operating system design by which the functionality of the system is moved out of the traditional "kernel", into a set of "servers" that communicate through a "minimal" kernel, leaving as little as possible in "system space" and as much as possible in "user space". A microkernel that is designed for a specific platform or device is only ever going to have what it needs to operate. The microkernel approach consists of defining a simple abstraction over the hardware, with a set of primitives or system calls to implement minimal OS services

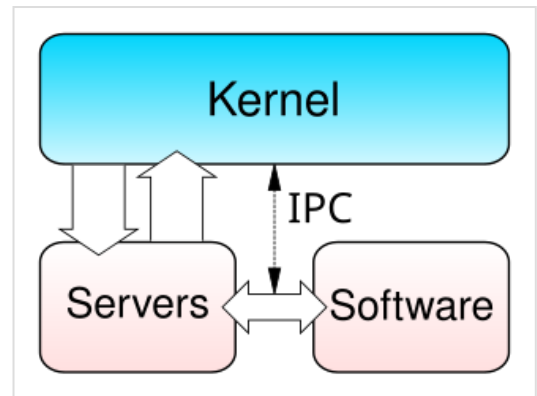
such as memory management, multitasking, and inter-process communication. Other services, including those normally provided by the kernel, such as networking, are implemented in user-space programs, referred to as *servers*. Microkernels are easier to maintain than monolithic kernels, but the large number of system calls and context switches might slow down the system because they typically generate more overhead than plain function calls.

Only parts that really require being in a privileged mode are in kernel space, such as IPC (Inter-Process Communication), a basic scheduler or scheduling primitives, basic memory handling, and basic I/O primitives. Many critical parts are now running in user space, including the complete scheduler, memory handling, file systems, and network stacks. Microkernels were invented as a reaction to traditional "monolithic" kernel design, whereby all system functionality was put in one static program running in a special "system" mode of the processor. In the microkernel, only the most fundamental of tasks are performed at this level, such as being able to access some (not necessarily all) of the hardware, manage memory and coordinate message passing between the processes. Some systems that use microkernels are QNX and GNU Hurd. In the case of QNX and GNU Hurd, user sessions can be entire snapshots of the system itself, or "views" as it is referred to. The very essence of the microkernel architecture illustrates some of its advantages:

- Easier to maintain
- Patches can be tested in a separate instance, and then swapped in to take over a production instance.
- Rapid development time and new software can be tested without having to reboot the kernel.
- More persistence in general: if one instance goes haywire, it is often possible to substitute it with an operational mirror.

Most microkernels use a message passing system to handle requests from one server to another. The message passing system generally operates on a port basis with the microkernel. As an example, if a request for more memory is sent, a port is opened with the microkernel and the request sent through. Once within the microkernel, the steps are similar to system calls. The rationale was that it would bring modularity in the system architecture, which would entail a cleaner system that is easier to debug or dynamically modify, customizable to users' needs, and higher performing. Microkernels are part of operating systems like GNU Hurd, MINIX, MkLinux, QNX and Redox OS.

Although microkernels are very small by themselves, in combination with all their required auxiliary code they are, in fact, often larger than monolithic kernels. Advocates of monolithic kernels also point out that the two-tiered structure of microkernel systems, in which most of the operating system does not interact directly with the hardware, creates a not-insignificant cost in terms of system efficiency. These types of kernels normally provide only the minimal services such as defining memory address spaces, inter-process communication (IPC) and process management. Other functions, such as running hardware processes, are not handled directly by microkernels. Proponents of microkernels point out those



In the microkernel approach, the kernel itself only provides basic functionality that allows the execution of servers, separate programs that assume former kernel functions, such as device drivers, GUI servers, etc.

monolithic kernels have the disadvantage that an error in the kernel can cause the entire system to crash. With a microkernel, however, if a kernel process crashes, it is still possible to prevent a crash of the system as a whole by merely restarting the service that caused the error.

Other services provided by the kernel such as networking are implemented in user-space programs referred to as *servers*. Servers allow the operating system to be modified by simply starting and stopping programs. For a machine without networking support, for instance, the networking server is not started. The task of moving in and out of the kernel to move data between the various applications and servers creates overhead that is detrimental to the efficiency of microkernels in comparison with monolithic kernels.

Disadvantages in the microkernel exist, however. Some are:

- Larger running memory footprint
- More software for interfacing is required, so there is a potential for performance loss.
- Messaging bugs can be harder to fix due to the longer trip they have to take, versus the one-off copy in a monolithic kernel.
- Process management in general can be very complicated.

The disadvantages for microkernels are extremely context-based. As an example, they work well for small single-purpose (and critical) systems because if not many processes need to run, then the complications of process management are effectively mitigated.

A microkernel allows the implementation of the remaining part of the operating system as programs running in user mode, and the use of different operating systems on top of the same unchanged kernel. It is also possible to dynamically switch among operating systems and to have more than one active simultaneously.^[28]

Monolithic kernels vs. microkernels

As the computer kernel grows, so grows the size and vulnerability of its trusted computing base, in addition to its memory footprint. This is mitigated to some degree by perfecting a virtual memory system, but not all computer architectures have virtual memory support.^[f] To reduce the kernel's footprint, extensive editing has to be performed to carefully remove unneeded code, which can be very difficult with non-obvious interdependencies between parts of a kernel with millions of lines of code.

By the early 1990s, due to the various shortcomings of monolithic kernels versus microkernels, monolithic kernels were considered obsolete by virtually all operating system researchers. As a result, the design of Linux as a monolithic kernel rather than a microkernel was the topic of a famous debate between Linus Torvalds and Andrew Tanenbaum.^[38] There is merit on both sides of the argument presented in the Tanenbaum–Torvalds debate.

Performance

Monolithic kernels are designed to have all of their code in the same address space (kernel space), which some developers argue is necessary to increase the performance of the system.^[39] Some developers also maintain that monolithic systems are extremely efficient if well written.^[39] The monolithic model tends to be more efficient^[40] through the use of shared kernel memory, rather than the slower IPC system of microkernel designs, which is typically based on message passing.

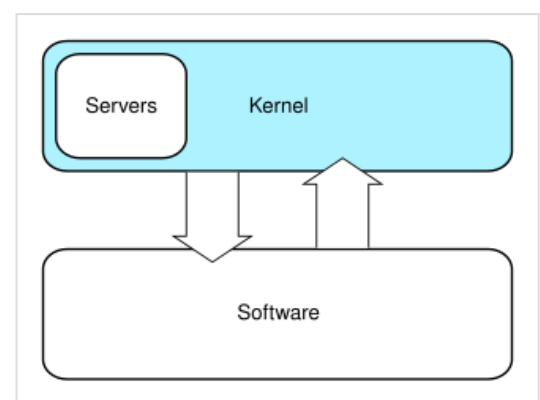
The performance of microkernels was poor in both the 1980s and early 1990s.^{[41][42]} Studies that empirically measured the performance of these microkernels did not analyze the reasons of such inefficiency.^[41] The explanations of this data were left to "folklore", with the assumption that they were due to the increased frequency of switches from "kernel-mode" to "user-mode", to the increased frequency of inter-process communication and to the increased frequency of context switches.^[41]

In fact, as guessed in 1995, the reasons for the poor performance of microkernels might as well have been: (1) an actual inefficiency of the whole microkernel *approach*, (2) the particular *concepts* implemented in those microkernels, and (3) the particular *implementation* of those concepts. Therefore it remained to be studied if the solution to build an efficient microkernel was, unlike previous attempts, to apply the correct construction techniques.^[41]

On the other end, the hierarchical protection domains architecture that leads to the design of a monolithic kernel^[36] has a significant performance drawback each time there's an interaction between different levels of protection (i.e., when a process has to manipulate a data structure both in "user mode" and "supervisor mode"), since this requires message copying by value.^[43]

Hybrid (or modular) kernels

Hybrid kernels are used in some commercial operating systems, including most versions of Microsoft Windows to date (NT 3.1, NT 3.5, NT 3.51, NT 4.0, 2000, XP, Vista, 7, 8, 8.1, 10 and 11). Apple's macOS uses a hybrid kernel called XNU, which is based upon code from OSF/1's Mach kernel (OSFMK 7.3)^[44] and FreeBSD's monolithic kernel. Hybrid kernels are similar to microkernels, except they include some additional code in kernel-space to increase performance. These kernels represent a compromise that was implemented by some developers to accommodate the major advantages of both monolithic and microkernels. These types of kernels are extensions of microkernels with some properties of monolithic kernels. Unlike monolithic kernels, these types of kernels are unable to load modules at runtime on their own. This implies running some services (such as the network stack or the filesystem) in kernel space to reduce the performance overhead of a traditional microkernel, but still running kernel code (such as device drivers) as servers in user space.



The hybrid kernel approach combines the speed and simpler design of a monolithic kernel with the modularity and execution safety of a microkernel.

Many traditionally monolithic kernels support loadable kernel modules. The most well known of these kernels is the Linux kernel. The modular kernel essentially can have parts of it that are built into the core kernel binary or binaries that load into memory on demand. A code-tainted module has the potential to destabilize a running kernel. By contrast, it is possible to write a driver for a microkernel in a completely separate memory space and test it before "going" live. When a kernel module is loaded, it accesses the monolithic portion's memory space by adding to it what it needs, therefore opening the doorway to possible pollution. A few advantages to the modular (or) hybrid kernel are:

- Faster development time for drivers that can operate from within modules. No reboot required for testing (provided the kernel is not destabilized).

- On demand capability versus spending time recompiling a whole kernel for things like new drivers or subsystems.
- Faster integration of third party technology (related to development but pertinent unto itself nonetheless).

Modules, generally, communicate with the kernel using a module interface of some sort. The interface is generalized (although particular to a given operating system) so it is not always possible to use modules. Often the device drivers may need more flexibility than the module interface affords. Essentially, it is two system calls and often the safety checks that only have to be done once in the monolithic kernel now may be done twice. Some of the disadvantages of the modular approach are:

- With more interfaces to pass through, the possibility of increased bugs exists (which implies more security holes).
- Maintaining modules can be confusing for some administrators when dealing with problems like symbol differences.

Nanokernels

A nanokernel delegates virtually all services – including even the most basic ones like interrupt controllers or the timer – to device drivers to make the kernel memory requirement even smaller than a traditional microkernel.^[45]

Exokernels

Exokernels are a still-experimental approach to operating system design. They differ from other types of kernels in limiting their functionality to the protection of and multiplexing of access to the raw hardware, providing no hardware abstractions on top of which to develop applications. This separation of hardware protection from hardware management enables application developers to determine how to make the most efficient use of the available hardware for each specific program.

Exokernels in themselves are extremely small. Nonetheless, they are accompanied by library operating systems (see also unikernel), providing application developers with the functionalities of a conventional operating system. A major advantage of exokernel-based systems is that they can incorporate multiple library operating systems, each exporting a different API, for example one for high level UI development and one for real-time control.

Multikernels

A multikernel operating system treats a multi-core machine as a network of independent cores, as if it were a distributed system. It does not assume shared memory but rather implements inter-process communications as message passing.^{[46][47]} Barrelfish was the first operating system to be described as a multikernel.

History of kernel development

Early operating system kernels

Strictly speaking, an operating system (and thus, a kernel) is not *required* to run a computer. Programs can be directly loaded and executed on the "bare metal" machine, provided that the authors of those programs are willing to work without any hardware abstraction or operating system support. Most early computers operated this way during the 1950s and early 1960s, which were reset and reloaded between the execution of different programs. Eventually, small ancillary programs such as program loaders and debuggers were left in memory between runs, or loaded from ROM. As these were developed, they formed the basis of what became early operating system kernels. The "bare metal" approach is still used today on some video game consoles and embedded systems,^[48] but in general, newer computers use modern operating systems and kernels.

In 1969, the RC 4000 Multiprogramming System introduced the system design philosophy of a small nucleus "upon which operating systems for different purposes could be built in an orderly manner",^[49] what would be called the microkernel approach.

Time-sharing operating systems

In the decade preceding Unix, computers had grown enormously in power – to the point where computer operators were looking for new ways to get people to use their spare time on their machines. One of the major developments during this era was time-sharing, whereby a number of users would get small slices of computer time, at a rate at which it appeared they were each connected to their own, slower, machine.^[50]

The development of time-sharing systems led to a number of problems. One was that users, particularly at universities where the systems were being developed, wanted to hack the system to get more CPU time. For this reason, security and access control became a major focus of the Multics project in 1965.^[51] Another ongoing issue was properly handling computing resources: users spent most of their time staring at the terminal and thinking about what to input instead of actually using the resources of the computer, and a time-sharing system should give the CPU time to an active user during these periods. Finally, the systems typically offered a memory hierarchy several layers deep, and partitioning this expensive resource led to major developments in virtual memory systems.

Amiga

The Commodore Amiga was released in 1985, and was among the first – and certainly most successful – home computers to feature an advanced kernel architecture. The AmigaOS kernel's executive component, *exec.library*, uses a microkernel message-passing design, but there are other kernel components, like *graphics.library*, that have direct access to the hardware. There is no memory protection, and the kernel is almost always running in user mode. Only special actions are executed in kernel mode, and user-mode applications can ask the operating system to execute their code in kernel mode.

Unix

During the design phase of Unix, programmers decided to model every high-level device as a file, because they believed the purpose of computation was data transformation.^[52]

For instance, printers were represented as a "file" at a known location – when data was copied to the file, it printed out. Other systems, to provide a similar functionality, tended to virtualize devices at a lower level – that is, both devices *and* files would be instances of some lower level concept. Virtualizing the

system at the file level allowed users to manipulate the entire system using their existing file management utilities and concepts, dramatically simplifying operation. As an extension of the same paradigm, Unix allows programmers to manipulate files using a series of small programs, using the concept of pipes, which allowed users to complete operations in stages, feeding a file through a chain of single-purpose tools. Although the end result was the same, using smaller programs in this way dramatically increased flexibility as well as ease of development and use, allowing the user to modify their workflow by adding or removing a program from the chain.



A diagram of the predecessor/successor family relationship for Unix-like systems

In the Unix model, the *operating system* consists of two parts: first, the huge collection of utility programs that drive most operations; second, the kernel that runs the programs.^[52] Under Unix, from a programming standpoint, the distinction between the two is fairly thin; the kernel is a program, running in supervisor mode,^[g] that acts as a program loader and supervisor for the small utility programs making up the rest of the system, and to provide locking and I/O services for these programs; beyond that, the kernel did not intervene at all in user space.

Over the years the computing model changed, and Unix's treatment of everything as a file or byte stream no longer was as universally applicable as it was before. Although a terminal could be treated as a file or a byte stream, which is printed to or read from, the same did not seem to be true for a graphical user interface. Networking posed another problem. Even if network communication can be compared to file access, the low-level packet-oriented architecture dealt with discrete chunks of data and not with whole files. As the capability of computers grew, Unix became increasingly cluttered with code. It is also because the modularity of the Unix kernel is extensively scalable.^[53] While kernels might have had 100,000 lines of code in the seventies and eighties, kernels like Linux, of modern Unix successors like GNU, have more than 13 million lines.^[54]

Modern Unix-derivatives are generally based on module-loading monolithic kernels. Examples of this are the Linux kernel in the many distributions of GNU, IBM AIX, as well as the Berkeley Software Distribution variant kernels such as FreeBSD, DragonFly BSD, OpenBSD, NetBSD, and macOS. Apart from these alternatives, amateur developers maintain an active operating system development community, populated by self-written hobby kernels that mostly end up sharing many features with Linux, FreeBSD, DragonflyBSD, OpenBSD, or NetBSD kernels and/or being compatible with them.^[55]

Classic Mac OS and macOS

Apple first launched its classic Mac OS in 1984, bundled with its Macintosh personal computer. Apple moved to a nanokernel design in Mac OS 8.6. Against this, the modern macOS (originally named Mac OS X) is based on Darwin, which uses a hybrid kernel called XNU, which was created by combining the 4.3BSD kernel and the Mach kernel.^[56]

Microsoft Windows

Microsoft Windows was first released in 1985 as an add-on to MS-DOS. Because of its dependence on another operating system, initial releases of Windows, prior to Windows 95, were considered an operating environment (not to be confused with an operating system). This product line continued to evolve through the 1980s and 1990s, with the Windows 9x series adding 32-bit addressing and pre-emptive multitasking; but ended with the release of Windows Me in 2000.

Microsoft also developed Windows NT, an operating system with a very similar interface, but intended for high-end and business users. This line started with the release of Windows NT 3.1 in 1993, and was introduced to general users with the release of Windows XP in October 2001—replacing Windows 9x with a completely different, much more sophisticated operating system. This is the line that continues with Windows 11.

The architecture of Windows NT's kernel is considered a hybrid kernel because the kernel itself contains tasks such as the Window Manager and the IPC Managers, with a client/server layered subsystem model.^[57] It was designed as a modified microkernel, as the Windows NT kernel was influenced by the Mach microkernel but does not meet all of the criteria of a pure microkernel.

IBM Supervisor

Supervisory program or supervisor is a computer program, usually part of an operating system, that controls the execution of other routines and regulates work scheduling, input/output operations, error actions, and similar functions and regulates the flow of work in a data processing system.

Historically, this term was essentially associated with IBM's line of mainframe operating systems starting with OS/360. In other operating systems, the supervisor is generally called the kernel.

In CP-40 and CP-67, IBM further abstracted the supervisor state from the hardware, resulting in a hypervisor that enabled full virtualization, i.e. the capacity to run multiple operating systems on the same machine totally independent of each other.

In the 1970s, IBM updated CP/CMS for the System/370 under the name Virtual Machine Facility/370 (VM/370). Hence the first such system was called *Virtual Machine* or VM.

Development of microkernels

Although Mach, developed by Richard Rashid at Carnegie Mellon University, is the best-known general-purpose microkernel, other microkernels have been developed with more specific aims. The L4 microkernel family (mainly the L3 and the L4 kernel) was created to demonstrate that microkernels are not necessarily slow.^[58] Newer implementations such as Fiasco and Pistachio are able to run Linux next to other L4 processes in separate address spaces.^{[59][60]}

Additionally, QNX is a microkernel that is principally used in embedded systems,^[61] and the open-source software MINIX, while originally created for educational purposes, is now focused on being a highly reliable and self-healing microkernel OS.

See also

- [Comparison of operating system kernels](#)
- [Inter-process communication](#)
- [Operating system](#)
- [Virtual memory](#)

Notes

- a. It may depend on the [Computer architecture](#)
- b. There is also [Derail \(DRL\)](#), but that has a restricted role.
- c. E.g., [Master mode entry](#)^[8] (MME) on the [GE-600 series](#),^[b] [Supervisor call](#)^[9] (SVC) on the [IBM System/360](#).
- d. E.g., [Unimplemented User Operation](#)^[10] (UUO) on the [DEC PDP-10](#).
- e. Some architectures have an opcode^[c] or block of opcodes^[d] dedicated to generating an interrupt. Some operating systems re-purpose an interrupt, e.g., [CP-67](#) and [VM](#) use a privileged instruction interrupt from a [Diagnose](#) instruction.
- f. Virtual addressing is most commonly achieved through a built-in [memory management unit](#).
- g. The highest privilege level has various names throughout different architectures, such as [supervisor mode](#), [kernel mode](#), [CPL0](#), [DPL0](#), [ring 0](#), etc. See [Protection ring](#) for more information.

References

1. "Kernel" (<http://www.linfo.org/kernel.html>). *Linfo*. Bellevue Linux Users Group. Archived (<http://web.archive.org/web/20061208185439/http://www.linfo.org/kernel.html>) from the original on 8 December 2006. Retrieved 15 September 2016.
2. Randal E. Bryant; David R. O'Hallaron (2016). *Computer Systems: A Programmer's Perspective* (Third ed.). Pearson. p. 17. ISBN 978-0-13-409266-9.
3. cf. [Daemon \(computing\)](#)
4. Roch 2004
5. Wulf 1974 pp.337–345
6. Silberschatz 1991
7. Tanenbaum, Andrew S. (2008). *Modern Operating Systems* (3rd ed.). Prentice Hall. pp. 50–51. ISBN 978-0-13-600663-3. ". . . nearly all system calls [are] invoked from C programs by calling a library procedure . . . The library procedure . . . executes a TRAP instruction to switch from user mode to kernel mode and start execution . . ."
8. "Miscellaneous Operations" (https://bitsavers.org/pdf/ge/GE-6xx/CPB-1004F_GE-635_Prog_Ref_Man_196907.pdf#page=145) (PDF). *GE-625/635 Programming Reference Manual* (https://bitsavers.org/pdf/ge/GE-6xx/CPB-1004F_GE-635_Prog_Ref_Man_196907.pdf) (PDF). Rev. April 1968. General Electric. July 1964. p. 132 (https://bitsavers.org/pdf/ge/GE-6xx/CPB-1004F_GE-635_Prog_Ref_Man_196907.pdf#page=147). CPB-1004F. Retrieved July 7, 2026.

9. "Supervisor Call" (https://bitsavers.org/pdf/ibm/360/princOps/A22-6821-7_360PrincOpsDec67.pdf#page=145) (PDF). *IBM System/360 Principles of Operation* (https://bitsavers.org/pdf/ibm/360/princOps/A22-6821-7_360PrincOpsDec67.pdf) (PDF). Systems Reference Library (Eighth ed.). IBM. September 1968. p. 74 (https://bitsavers.org/pdf/ibm/360/princOps/A22-6821-7_360PrincOpsDec67.pdf#page=84). A22-6821-7. Retrieved July 7, 2026.
10. "2.16 Unimplemented Operations" (https://bitsavers.org/pdf/dec/pdp10/1982_ProcRefMan.pdf#page=183) (PDF). *DECsystem-10 - DECSYSTEM-20 - Processor Reference Manual - AA-H391A-TK, AM-4391A-T1* (https://bitsavers.org/pdf/dec/pdp10/1982_ProcRefMan.pdf) (PDF). June 1982. pp. 2-123 – 2-127. AD-H391A. Retrieved July 7, 2026.
11. Denning 1976
12. Swift 2005, p.29 quote: "isolation, resource control, decision verification (checking), and error recovery."
13. Schroeder 72
14. Linden 76
15. Eranian, Stephane; Mosberger, David (2002). "Virtual Memory in the IA-64 Linux Kernel" (<http://www.informit.com/articles/article.aspx?p=29961>). *IA-64 Linux Kernel: Design and Implementation*. Prentice Hall PTR. ISBN 978-0-13-061014-0.
16. Silberschatz & Galvin, Operating System Concepts, 4th ed, pp. 445 & 446
17. Hoch, Charles; J. C. Browne (July 1980). "An implementation of capabilities on the PDP-11/45" (<https://doi.org/10.1145%2F850697.850701>). *ACM SIGOPS Operating Systems Review*. **14** (3): 22–32. doi:10.1145/850697.850701 (<https://doi.org/10.1145%2F850697.850701>). S2CID 17487360 (<https://api.semanticscholar.org/CorpusID:17487360>).
18. Schneider, Fred B.; Morrisett, Greg; Harper, Robert. "A Language-Based Approach to Security" (<https://www.cs.cmu.edu/~rwh/papers/langsec/dagstuhl.pdf>) (PDF). Archived (<http://web.archive.org/web/20181222193622/http://www.cs.cmu.edu/~rwh/papers/langsec/dagstuhl.pdf>) (PDF) from the original on 2018-12-22.
19. Loscocco, P. A.; Smalley, S. D.; Muckelbauer, P. A.; Taylor, R. C.; Turner, S. J.; Farrell, J. F. (October 1998). "The Inevitability of Failure: The Flawed Assumption of Security in Modern Computing Environments" (<https://web.archive.org/web/20070621155813/http://jya.com/paperF1.htm>). *Proceedings of the 21st National Information Systems Security Conference*. pp. 303–314. Archived from the original (<http://www.jya.com/paperF1.htm>) on 2007-06-21.
20. Lepreau, Jay; Ford, Bryan; Hibler, Mike (1996). "The persistent relevance of the local operating system to global applications". *Proceedings of the 7th workshop on ACM SIGOPS European workshop Systems support for worldwide applications - EW 7*. pp. 133–140. doi:10.1145/504450.504477 (<https://doi.org/10.1145%2F504450.504477>). ISBN 978-1-4503-7339-5. S2CID 10027108 (<https://api.semanticscholar.org/CorpusID:10027108>).
21. Anderson, J. (October 1972). Computer Security Technology Planning Study (<http://csrc.nist.gov/publications/history/ande72.pdf>) (PDF) (Report). Vol. II. Air Force Electronic Systems Division. ESD-TR-73-51, Vol. II. Archived (<https://web.archive.org/web/20110721060319/http://csrc.nist.gov/publications/history/ande72.pdf>) (PDF) from the original on 2011-07-21.
22. Jerry H. Saltzer; Mike D. Schroeder (September 1975). "The protection of information in computer systems" (<http://web.mit.edu/Saltzer/www/publications/protection/>). *Proceedings of the IEEE*. **63** (9): 1278–1308. Bibcode:1975IEEEP..63.1278S (<https://ui.adsabs.harvard.edu/abs/1975IEEEP..63.1278S>). CiteSeerX 10.1.1.126.9257 (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.126.9257>). doi:10.1109/PROC.1975.9939 (<https://doi.org/10.1109%2FPROC.1975.9939>). S2CID 269166 (<https://api.semanticscholar.org/CorpusID:269166>). Archived (<https://web.archive.org/web/20210308030942/http://web.mit.edu/Saltzer/www/publications/protection/>) from the original on 2021-03-08. Retrieved 2007-07-15.
23. "Fine-grained kernel isolation" (<https://mars-research.github.io/projects/kernel-isolation/>). *mars-research.github.io*. Retrieved 15 September 2022.

24. Fetzer, Mary. "Automatic device driver isolation protects against bugs in operating systems" (<https://techxplore.com/news/2022-08-automatic-device-driver-isolation-bugs.html>). *Pennsylvania State University via techxplore.com*. Retrieved 15 September 2022.
25. Huang, Yongzhe; Narayanan, Vikram; Detweiler, David; Huang, Kaiming; Tan, Gang; Jaeger, Trent; Burtsev, Anton (2022). "KSplitt: Automating Device Driver Isolation" (https://www.cse.psu.edu/~gxt29/papers/ksplit_osdi22.pdf) (PDF). Retrieved 20 December 2023.
26. Jonathan S. Shapiro; Jonathan M. Smith; David J. Farber (1999). "EROS". *ACM SIGOPS Operating Systems Review*. **33** (5): 170–185. doi:10.1145/319344.319163 (<https://doi.org/10.1145/319344.319163>).
27. Dijkstra, E. W. (September 1965). "Co-operating sequential processes" (<https://pure.tue.nl/ws/files/4279816/344354178746665.pdf>) (PDF). In F. Genuys (ed.). *Programming languages: NATO Advanced Study Institute: lectures given at a three weeks Summer School held in Villard-le-Lans, 1966*. Academic Press Inc.. Math. Dep., Technological U., Eindhoven. pp. 43–112.
28. Brinch Hansen 70 pp.238–241
29. Harrison, M. C.; Schwartz, J. T. (1967). "SHARER, a time sharing system for the CDC 6600" (<https://doi.org/10.1145/2F363717.363778>). *Communications of the ACM*. **10** (10): 659–665. doi:10.1145/363717.363778 (<https://doi.org/10.1145/2F363717.363778>). S2CID 14550794 (<https://api.semanticscholar.org/CorpusID:14550794>).
30. Huxtable, D. H. R.; Warwick, M. T. (1967). "Dynamic Supervisors - their design and construction". *Proceedings of the ACM symposium on Operating System Principles - SOSOP '67*. pp. 11.1 – 11.17. doi:10.1145/800001.811675 (<https://doi.org/10.1145/2F800001.811675>). ISBN 978-1-4503-7370-8. S2CID 17709902 (<https://api.semanticscholar.org/CorpusID:17709902>).
31. Baiardi 1988
32. Levin 75
33. Denning 1980
34. Nehmer, Jürgen (1991). "The Immortality of Operating Systems, or: Is Research in Operating Systems still Justified?" (<https://link.springer.com/chapter/10.1007/2FBFb0024528#page-1>). *Lecture Notes In Computer Science; Vol. 563. Proceedings of the International Workshop on Operating Systems of the 90s and Beyond*. pp. 77–83. doi:10.1007/BFb0024528 (<https://doi.org/10.1007/2FBFb0024528>). ISBN 3-540-54987-0. "The past 25 years have shown that research on operating system architecture had a minor effect on existing main stream [*sic*] systems."
35. Levy 84, p.1 quote: "Although the complexity of computer applications increases yearly, the underlying hardware architecture for applications has remained unchanged for decades."
36. Levy 84, p.1 quote: "Conventional architectures support a single privileged mode of operation. This structure leads to monolithic design; any module needing protection must be part of the single operating system kernel. If, instead, any module could execute within a protected domain, systems could be built as a collection of independent modules extensible by any user."
37. "Open Sources: Voices from the Open Source Revolution" (<https://www.oreilly.com/openbook/opensources/book/appa.html>). 1-56592-582-3. 29 March 1999. Archived (<https://web.archive.org/web/20200201103745/https://www.oreilly.com/openbook/opensources/book/appa.html>) from the original on 1 February 2020. Retrieved 24 March 2019.

38. Recordings of the debate between Torvalds and Tanenbaum can be found at dina.dk (http://www.dina.dk/~abraham/Linus_vs_Tanenbaum.html) Archived (https://web.archive.org/web/20121003060514/http://www.dina.dk/~abraham/Linus_vs_Tanenbaum.html) 2012-10-03 at the Wayback Machine, groups.google.com (https://groups.google.com/group/comp.os.minix/browse_thread/thread/c25870d7a41696d2/f447530d082cd95d?vc=2#f447530d082cd95d) Archived (https://web.archive.org/web/20130526014635/http://groups.google.com/group/comp.os.minix/browse_thread/thread/c25870d7a41696d2/f447530d082cd95d?vc=2#f447530d082cd95d) 2013-05-26 at the Wayback Machine, oreilly.com (<https://www.oreilly.com/catalog/opensources/book/appa.html>) Archived (<https://web.archive.org/web/20140921050425/http://oreilly.com/catalog/opensources/book/appa.html>) 2014-09-21 at the Wayback Machine and Andrew Tanenbaum's website (<http://www.cs.vu.nl/~ast/reliable-os/>) Archived (<https://web.archive.org/web/20150805132304/http://www.cs.vu.nl/~ast/reliable-os/>) 2015-08-05 at the Wayback Machine
39. Matthew Russell. "What Is Darwin (and How It Powers Mac OS X)" (<https://web.archive.org/web/20071208130117/http://www.oreilly.com/pub/a/mac/2005/09/27/what-is-darwin.html?page=2>). O'Reilly Media. Archived from the original (<http://oreilly.com/pub/a/mac/2005/09/27/what-is-darwin.html?page=2>) on 2007-12-08. Retrieved 2008-12-09. "The tightly coupled nature of a monolithic kernel allows it to make very efficient use of the underlying hardware [...] Microkernels, on the other hand, run a lot more of the core processes in userland. [...] Unfortunately, these benefits come at the cost of the microkernel having to pass a lot of information in and out of the kernel space through a process known as a context switch. Context switches introduce considerable overhead and therefore result in a performance penalty."
40. "Operating Systems/Kernel Models - Wikiversity" (https://en.wikiversity.org/wiki/Operating_Systems/Kernel_Models). *en.wikiversity.org*. Archived (https://web.archive.org/web/20141218090736/https://en.wikiversity.org/wiki/Operating_Systems/Kernel_Models) from the original on 2014-12-18. Retrieved 2014-12-18.
41. Liedtke 95
42. Härtig 97
43. Hansen 73, section 7.3 p.233 "*interactions between different levels of protection require transmission of messages by value*"
44. Magee, Jim. *WWDC 2000 Session 106 – Mac OS X: Kernel* (<https://ghostarchive.org/varchive/youtube/20211030/ggnFoDqzGMU>). 14 minutes in. Archived from the original (<https://www.youtube.com/watch?v=ggnFoDqzGMU>) on 2021-10-30.
45. "KeyKOS Nanokernel Architecture" (<https://web.archive.org/web/20110621235229/http://www.cis.upenn.edu/~KeyKOS/NanoKernel/NanoKernel.html>). Archived from the original (<http://www.cis.upenn.edu/~KeyKOS/NanoKernel/NanoKernel.html>) on 2011-06-21.
46. Baumann, Andrew; Barham, Paul; Dagand, Pierre-Evariste; Harris, Tim; Isaacs, Rebecca; Peter, Simon; Roscoe, Timothy; Schüpbach, Adrian; Singhanian, Akhilesh (2009). *The Multikernel: a new OS architecture for scalable multicore systems* (<https://www.sigops.org/s/conferences/sosp/2009/papers/baumann-sosp09.pdf>) (PDF). 22nd Symposium on Operating Systems Principles.
47. "The Barrelfish operating system" (<https://barrelfish.org>).
48. Ball: Embedded Microprocessor Designs, p. 129
49. Hansen 2001 (os), pp.17–18
50. "BSTJ version of C.ACM Unix paper" (<http://cm.bell-labs.com/cm/cs/who/dmr/cacm.html>). *bell-labs.com*. Archived (<https://web.archive.org/web/20051230143815/http://cm.bell-labs.com/cm/cs/who/dmr/cacm.html>) from the original on 2005-12-30. Retrieved 2006-08-17.
51. Corbató, F. J.; Vissotsky, V. A. *Introduction and Overview of the Multics System* (<http://www.multicians.org/fjcc1.html>). 1965 Fall Joint Computer Conference. Archived (<https://web.archive.org/web/20110709025836/http://www.multicians.org/fjcc1.html>) from the original on 2011-07-09.

52. "The Single Unix Specification" (https://www.unix.org/what_is_unix/single_unix_specification.html). The Open Group. Archived (https://web.archive.org/web/20161004051725/http://www.unix.org/what_is_unix/single_unix_specification.html) from the original on 2016-10-04. Retrieved 2016-09-29.
53. "Unix's Revenge" (<http://www.asymco.com/2010/09/29/unixs-revenge/>). *asymco.com*. 29 September 2010. Archived (<https://web.archive.org/web/20101109010117/http://www.asymco.com/2010/09/29/unixs-revenge/>) from the original on 9 November 2010. Retrieved 2 October 2010.
54. Wheeler, David A. (October 12, 2004). "Linux Kernel 2.6: It's Worth More!" (<https://dwheeler.com/essays/linux-kernel-cost.html>).
55. This community mostly gathers at Bona Fide OS Development (<http://www.osdever.net>) Archived (<https://web.archive.org/web/20220117235428/http://www.osdever.net/>) 2022-01-17 at the Wayback Machine, The Mega-Tokyo Message Board (<http://www.mega-tokyo.com/forum>) Archived (<https://web.archive.org/web/20220125090706/https://mega-tokyo.com/blog/>) 2022-01-25 at the Wayback Machine and other operating system enthusiast web sites.
56. Singh, Amit (December 2003). "XNU: The Kernel" (https://web.archive.org/web/20110812000315/http://osxbook.com/book/bonus/ancient/whatismacosx/arch_xnu.html). Archived from the original (http://www.kernelthread.com/mac/osx/arch_xnu.html) on 2011-08-12.
57. "Windows - Official Site for Microsoft Windows 10 Home & Pro OS, laptops, PCs, tablets & more" (<https://windows.com/>). *windows.com*. Archived (<https://web.archive.org/web/20110820081607/http://www.windows.com/>) from the original on 2011-08-20. Retrieved 2019-03-24.
58. "The L4 microkernel family - Overview" (<http://os.inf.tu-dresden.de/L4/overview.html>). *os.inf.tu-dresden.de*. Archived (<https://web.archive.org/web/20060821060811/http://os.inf.tu-dresden.de/L4/overview.html>) from the original on 2006-08-21. Retrieved 2006-08-11.
59. "The Fiasco microkernel - Overview" (<http://os.inf.tu-dresden.de/fiasco/overview.html>). *os.inf.tu-dresden.de*. Archived (<https://web.archive.org/web/20060616081310/http://os.inf.tu-dresden.de/fiasco/overview.html>) from the original on 2006-06-16. Retrieved 2006-07-10.
60. Zoller (inaktiv), Heinz (7 December 2013). "L4Ka - L4Ka Project" (<http://www.l4ka.org/>). *www.l4ka.org*. Archived (<https://web.archive.org/web/20010419064321/http://www.l4ka.org/>) from the original on 19 April 2001. Retrieved 24 March 2019.
61. "QNX Operating Systems" (<http://blackberry.qnx.com/en/products/neutrino-rtos/index>). *blackberry.qnx.com*. Archived (<https://web.archive.org/web/20190324031005/http://blackberry.qnx.com/en/products/neutrino-rtos/index>) from the original on 2019-03-24. Retrieved 2019-03-24.

Sources

- Roch, Benjamin (2004). "Monolithic kernel vs. Microkernel" (https://web.archive.org/web/20061101012856/http://www.vmars.tuwien.ac.at/courses/akti12/journal/04ss/article_04ss_Roch.pdf) (PDF). Archived from the original (http://www.vmars.tuwien.ac.at/courses/akti12/journal/04ss/article_04ss_Roch.pdf) (PDF) on 2006-11-01. Retrieved 2006-10-12.
- Silberschatz, Abraham; James L. Peterson; Peter B. Galvin (1991). *Operating system concepts* (http://portal.acm.org/citation.cfm?id=95329&dl=acm&coll=&CFID=15151515&CF_TOKEN=6184618). Boston, Massachusetts: Addison-Wesley. p. 696. ISBN 978-0-201-51379-0.
- Ball, Stuart R. (2002) [2002]. *Embedded Microprocessor Systems: Real World Designs* (first ed.). Elsevier Science. ISBN 978-0-7506-7534-5.
- Deitel, Harvey M. (1984) [1982]. *An introduction to operating systems* (<https://archive.org/details/introductiontoopdeit00deit/page/673>) (revisited first ed.). Addison-Wesley. p. 673 (<http://s://archive.org/details/introductiontoopdeit00deit/page/673>). ISBN 978-0-201-14502-1.

- Denning, Peter J. (December 1976). "Fault tolerant operating systems". *ACM Computing Surveys*. **8** (4): 359–389. doi:10.1145/356678.356680 ([https://doi.org/10.1145/356678.356680](https://doi.org/10.1145%2F356678.356680)). ISSN 0360-0300 (<https://search.worldcat.org/issn/0360-0300>). S2CID 207736773 (<https://api.semanticscholar.org/CorpusID:207736773>).
- Denning, Peter J. (April 1980). "Why not innovations in computer architecture?" (<https://doi.org/10.1145%2F859504.859506>). *ACM SIGARCH Computer Architecture News*. **8** (2): 4–7. doi:10.1145/859504.859506 ([https://doi.org/10.1145/859504.859506](https://doi.org/10.1145%2F859504.859506)). ISSN 0163-5964 (<https://search.worldcat.org/issn/0163-5964>). S2CID 14065743 (<https://api.semanticscholar.org/CorpusID:14065743>).
- Hansen, Per Brinch (April 1970). "The nucleus of a Multiprogramming System". *Communications of the ACM*. **13** (4): 238–241. CiteSeerX 10.1.1.105.4204 (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.105.4204>). doi:10.1145/362258.362278 ([https://doi.org/10.1145/362258.362278](https://doi.org/10.1145%2F362258.362278)). ISSN 0001-0782 (<https://search.worldcat.org/issn/0001-0782>). S2CID 9414037 (<https://api.semanticscholar.org/CorpusID:9414037>).
- Hansen, Per Brinch (1973). *Operating System Principles* (<https://archive.org/details/operatingsystemp0000brin/page/496>). Englewood Cliffs: Prentice Hall. p. 496 (<https://archive.org/details/operatingsystemp0000brin/page/496>). ISBN 978-0-13-637843-3.
- Hansen, Per Brinch (2001). "The evolution of operating systems" (<http://brinch-hansen.net/papers/2001b.pdf>) (PDF). Archived (<https://web.archive.org/web/20110725214752/http://brinch-hansen.net/papers/2001b.pdf>) (PDF) from the original on 2011-07-25. Retrieved 2006-10-24. included in book: Per Brinch Hansen, ed. (2001). "1 The evolution of operating systems". *Classic operating systems: from batch processing to distributed systems* (<https://doi.org/10.5555/360596>). New York: Springer-Verlag. pp. 1–36. ISBN 978-0-387-95113-3.
- Härtig, Hermann; Hohmuth, Michael; Liedtke, Jochen; Schönberg, Sebastian; Wolter, Jean (October 5–8, 1997). "The performance of μ -kernel-based systems" (<http://os.inf.tu-dresden.de/pubs/sosp97/#Karshmer:1991:OSA>). *Proceedings of the sixteenth ACM symposium on Operating systems principles - SOSP '97*. 16th ACM Symposium on Operating Systems Principles (SOSP'97). Saint-Malo, France. doi:10.1145/268998.266660 ([https://doi.org/10.1145/268998.266660](https://doi.org/10.1145%2F268998.266660)). ISBN 978-0-89791-916-6. S2CID 1706253 (<https://api.semanticscholar.org/CorpusID:1706253>). Archived (<https://web.archive.org/web/20200217211614/http://os.inf.tu-dresden.de/pubs/sosp97/#Karshmer:1991:OSA>) from the original on 2020-02-17., Härtig, Hermann; Hohmuth, Michael; Liedtke, Jochen; Schönberg, Sebastian (December 1997). "The performance of μ -kernel-based systems" ([https://doi.org/10.1145/269005.266660](https://doi.org/10.1145%2F269005.266660)). *ACM SIGOPS Operating Systems Review*. **31** (5): 66–77. doi:10.1145/269005.266660 ([https://doi.org/10.1145/269005.266660](https://doi.org/10.1145%2F269005.266660)).
- Houdek, M. E.; Soltis, F. G.; Hoffman, R. L. (1981). "IBM System/38 support for capability-based addressing" (<http://portal.acm.org/citation.cfm?id=800052.801885>). *Proceedings of the 8th ACM International Symposium on Computer Architecture*. ACM/IEEE. pp. 341–348.
- *The IA-32 Architecture Software Developer's Manual, Volume 1: Basic Architecture* (<http://www.intel.com/design/pentium4/manuals/24547010.pdf>) (PDF). Intel Corporation. 2002.
- Levin, R.; Cohen, E.; Corwin, W.; Pollack, F.; Wulf, William (1975). "Policy/Mechanism separation in Hydra" ([https://doi.org/10.1145/1067629.806531](https://doi.org/10.1145%2F1067629.806531)). *ACM SIGOPS Operating Systems Review*. **9** (5): 132–140. doi:10.1145/1067629.806531 ([https://doi.org/10.1145/1067629.806531](https://doi.org/10.1145%2F1067629.806531)).
- Levy, Henry M. (1984). *Capability-based computer systems* (<http://www.cs.washington.edu/homes/levy/capabook/index.html>). Maynard, Mass: Digital Press. ISBN 978-0-932376-22-0. Archived (<https://web.archive.org/web/20070713175342/http://www.cs.washington.edu/homes/levy/capabook/index.html>) from the original on 2007-07-13. Retrieved 2007-07-18.

- Liedtke, Jochen (December 1995). "On μ -Kernel Construction" (<https://web.archive.org/web/20070313142519/http://i30www.ira.uka.de/research/publications/papers/index.php?lid=en&ocid=642>). *Proc. 15th ACM Symposium on Operating System Principles (SOSP)*. Archived from the original (<http://i30www.ira.uka.de/research/publications/papers/index.php?lid=en&ocid=642>) on 2007-03-13.
- Linden, Theodore A. (December 1976). "Operating System Structures to Support Security and Reliable Software". *ACM Computing Surveys*. **8** (4): 409–445. doi:10.1145/356678.356682 (<https://doi.org/10.1145%2F356678.356682>). hdl:2027/mdp.39015086560037 (<https://hdl.handle.net/2027%2Fmdp.39015086560037>). ISSN 0360-0300 (<https://search.worldcat.org/issn/0360-0300>). S2CID 16720589 (<https://api.semanticscholar.org/CorpusID:16720589>)., "Operating System Structures to Support Security and Reliable Software" (<https://csrc.nist.gov/files/pubs/conference/1998/10/08/proceedings-of-the-21st-nissc-1998/final/docs/early-cs-papers/lind76.pdf>) (PDF). Retrieved 2023-12-20.
- Lorin, Harold (1981). *Operating systems* (<https://archive.org/details/operatingsystems0000lori/page/161>). Boston, Massachusetts: Addison-Wesley. pp. 161–186 (<https://archive.org/details/operatingsystems0000lori/page/161>). ISBN 978-0-201-14464-2.
- Schroeder, Michael D.; Jerome H. Saltzer (March 1972). "A hardware architecture for implementing protection rings". *Communications of the ACM*. **15** (3): 157–170. CiteSeerX 10.1.1.83.8304 (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.83.8304>). doi:10.1145/361268.361275 (<https://doi.org/10.1145%2F361268.361275>). ISSN 0001-0782 (<https://search.worldcat.org/issn/0001-0782>). S2CID 14422402 (<https://api.semanticscholar.org/CorpusID:14422402>).
- Shaw, Alan C. (1974). *The logical design of Operating systems* (<https://archive.org/details/logicaldesignof00shaw/page/304>). Prentice-Hall. p. 304 (<https://archive.org/details/logicaldesignof00shaw/page/304>). ISBN 978-0-13-540112-5.
- Tanenbaum, Andrew S. (1979). *Structured Computer Organization*. Englewood Cliffs, New Jersey: Prentice-Hall. ISBN 978-0-13-148521-1.
- Wulf, W.; E. Cohen; W. Corwin; A. Jones; R. Levin; C. Pierson; F. Pollack (June 1974). "HYDRA: the kernel of a multiprocessor operating system" (<https://web.archive.org/web/20070926161655/http://www.cs.virginia.edu/papers/p337-wulf.pdf>) (PDF). *Communications of the ACM*. **17** (6): 337–345. doi:10.1145/355616.364017 (<https://doi.org/10.1145%2F355616.364017>). ISSN 0001-0782 (<https://search.worldcat.org/issn/0001-0782>). S2CID 8011765 (<https://api.semanticscholar.org/CorpusID:8011765>). Archived from the original (<http://www.cs.virginia.edu/papers/p337-wulf.pdf>) (PDF) on 2007-09-26. Retrieved 2007-07-18.
- Baiardi, F.; A. Tomasi; M. Vanneschi (1988). *Architettura dei Sistemi di Elaborazione, volume 1* (<https://web.archive.org/web/20120627162353/http://www.pangloss.it/libro.php?isbn=882042746X&id=4357&PHPSESSID=9da1895b18ed1cda115cf1c7ace9bdf0>) (in Italian). Franco Angeli. ISBN 978-88-204-2746-7. Archived from the original (<http://www.pangloss.it/libro.php?isbn=882042746X&id=4357&PHPSESSID=9da1895b18ed1cda115cf1c7ace9bdf0>) on 2012-06-27. Retrieved 2006-10-10.
- Swift, Michael M.; Brian N. Bershad; Henry M. Levy. *Improving the reliability of commodity operating systems* (<http://nooks.cs.washington.edu/nooks-tocs.pdf>) (PDF). Archived (<https://web.archive.org/web/20070719222901/http://nooks.cs.washington.edu/nooks-tocs.pdf>) (PDF) from the original on 2007-07-19. Retrieved 2007-07-16.
- Gettys, James; Karlton, Philip L.; McGregor, Scott (1990). "The X window system, version 11". *Software: Practice and Experience*. **20**: S35–S67. doi:10.1002/spe.4380201404 (<https://doi.org/10.1002%2Fspe.4380201404>). S2CID 26329062 (<https://api.semanticscholar.org/CorpusID:26329062>).

- Michael M. Swift; Brian N. Bershad; Henry M. Levy (February 2005). "Improving the reliability of commodity operating systems". *ACM Transactions on Computer Systems*. **23** (1). Association for Computing Machinery: 77–110. doi:10.1145/1047915.1047919 (<https://doi.org/10.1145/1047915.1047919>). eISSN 1557-7333 (<https://search.worldcat.org/issn/1557-7333>). ISSN 0734-2071 (<https://search.worldcat.org/issn/0734-2071>). S2CID 208013080 (<https://api.semanticscholar.org/CorpusID:208013080>).

Further reading

- Andrew S. Tanenbaum, Albert S. Woodhull, *Operating Systems: Design and Implementation (Third edition)*;
- Andrew S. Tanenbaum, Herbert Bos, *Modern Operating Systems (Fourth edition)*;
- Daniel P. Bovet, Marco Cesati, *Understanding the Linux Kernel (Third edition)*;
- David A. Patterson, John L. Hennessy, *Computer Organization and Design (Sixth edition)*, Morgan Kaufmann (ISBN 978-0-12-820109-1);
- B.S. Chalk, A.T. Carter, R.W. Hind, *Computer Organisation and Architecture: An Introduction (Second edition)*, Palgrave Macmillan (ISBN 978-1-4039-0164-4).

External links

- Kernel Comparison: Linux (2.6.22) versus Windows (Vista) (<https://web.archive.org/web/20080914072704/http://widefox.pbwiki.com/Kernel%20Comparison%20Linux%20vs%20Windows>) at the Wayback Machine (archived 2008-09-14)
 - The Barrelfish Operating System (<https://barrelfish.org/>)
-

Retrieved from "[https://en.wikipedia.org/w/index.php?title=Kernel_\(operating_system\)&oldid=1363169421](https://en.wikipedia.org/w/index.php?title=Kernel_(operating_system)&oldid=1363169421)"